COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:		**SECTION:**	
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Ace Tadalan	**INSTRUCTOR:**	Engr. Quejado

ACTIVITY NO. 4.1 Hands-on Activity: Ansible Basics

1. PROCEDURE OUTPUT

What procedures did you perform?

1. Locally, we use the command sudo apt update when we want to download package information from all configured resources. The sources are often defined in /etc/apt/sources.list file and other files located in /etc/apt/sources.list.d/ directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:
ansible all -m apt -a update_cache=true

What is the result of the command? Is it successful?
Try editing the command and add something that would elevate the privilege. Issue the command ansible all -m apt -a update_cache=true --become --ask-become-pass. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The update_cache=true is the same thing as running sudo apt update. The --become command elevates the privileges and the --ask-become-pass asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: ansible all -m apt -a name=vim-nox --become --ask-become-pass. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.
a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively. Was the command successful?
b. Check the logs in the servers using the following commands: cd /var/log. After this, issue the command ls, go to the folder apt and open history.log. Describe what you see in the history.log.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.
a. Issue the command: ansible all -m apt -a name=snapd --become --ask-become-pass
Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?
b. Now, try to issue this command: ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass
Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

```
ace@workstation:~/ansible-lab$ ansible all -m apt -a update
_cache=true -i inventory.yaml --become --ask-become-pass
BECOME password:
[WARNING]: Updating cache and auto-installing missing depen
dency: python3-apt
centos1 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "cmd": "apt-get update",
    "msg": "[Errno 2] No such file or directory: b'apt-get'
",
    "rc": 2,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "",
    "stdout_lines": []
}
server2 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787538988,
    "cache_updated": true,
    "changed": true
}
server1 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787538988,
```

```
ace@workstation:~/ansible-lab$ which vim
/usr/bin/vim
ace@workstation:~/ansible-lab$
```

```
server2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787538988,
    "cache_updated": false,
    "changed": false
}
server1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787538988,
    "cache_updated": false,
```

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command nano install_apache.yml. This will create a playbook file called install_apache.yaml. The .yaml is the basic standard extension for playbook files.
When the editor appears, type the following:
Make sure to save the file. Take note also of the alignments of the texts. Use TAB instead of SPACE BAR for spacing.

2. Run the yml file using the command: ansible-playbook --ask-become-pass install_apache.yml. Describe the result of this command.

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.

4. Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized. What is the output?

5. This time, we are going to put additional tasks into our playbook. Edit the install_apache.yml. As you can see, we are now adding an additional command, which is the update_cache. This command updates

existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

Save the changes to this file and exit.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers?

7. Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier.

Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository

```
ace@workstation:~/ansible-lab$ nano install_apache.yml
ace@workstation:~/ansible-lab$ ansible-playbook -i inventor
y.yaml install_apache.yml --ask-become-pass
BECOME password:

PLAY [ubuntu] ***********************************************
*********************

TASK [Gathering Facts] **************************************
*********************
ok: [server2]
ok: [server1]

TASK [install apache2 package] ******************************
*********************
fatal: [server2]: FAILED! => {"changed": false, "msg": "No
package matching 'totallyfakepackage123' is available"}
fatal: [server1]: FAILED! => {"changed": false, "msg": "No
package matching 'totallyfakepackage123' is available"}

PLAY RECAP **************************************************
*********************
server1                    : ok=1    changed=0    unreachab
le=0    failed=1    skipped=0    rescued=0    ignored=0
server2                    : ok=1    changed=0    unreachab
le=0    failed=1    skipped=0    rescued=0    ignored=0

ace@workstation:~/ansible-lab$
```

```
ace@workstation:~/ansible-lab$ ansible-playbook -i inventor
y.yaml install_apache.yml --ask-become-pass
BECOME password:

PLAY [ubuntu] ***********************************************
*********************

TASK [Gathering Facts] **************************************
*********************
ok: [server1]
ok: [server2]

TASK [update repository index] ******************************
*********************
changed: [server2]
changed: [server1]

TASK [install apache2 package] ******************************
*********************
ok: [server2]
ok: [server1]

PLAY RECAP **************************************************
*********************
server1                    : ok=3    changed=1    unreachab
le=0    failed=0    skipped=0    rescued=0    ignored=0
server2                    : ok=3    changed=1    unreachab
le=0    failed=0    skipped=0    rescued=0    ignored=0

ace@workstation:~/ansible-lab$
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. What is the importance of using a playbook?
   - This automates the tasks on many servers at once instead of manually doing them one by one. It provides efficiency, saves time and avoids unnecessary mistakes.

2. Summarize what we have done on this activity
   - We used Ansible to update, install, and check packages on remotely with the existing servers. Creating a playbook that installed Apache2 and PHP support. We also tested what happens with a wrong package name, then pushed our work to GitHub.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.